



The application of improved physics-informed neural network (IPINN) method in finance

Yuxing Bai · Temuer Chaolu · Sudao Bilige

Received: 23 August 2021 / Accepted: 12 December 2021 / Published online: 29 January 2022
© The Author(s), under exclusive licence to Springer Nature B.V. 2021

Abstract With the development of computers and neural networks, the traditional methods of solving differential equations have been greatly developed. Typical examples are the differential equations of population, finance, infectious disease and traffic problems solved by neural network method. Recently, the popular physics-informed neural network (PINN) method has been proved to be able to solve the numerical solution of PDEs. Based on the PINN method, this paper proposes an improved PINN method (IPINN) that is to introduce local adaptive activation function of neurons into PINN network to improve the performance of neural network and successfully applies the IPINN method to the Ivancevic option pricing model and Black–Scholes model in finance. The rogue wave solution and soliton solution of the Ivancevic option pricing model, and the numerical solution of the Black–Scholes model are solved, respectively. At the same time, it can be shown that the IPINN method has the characteristics of faster convergence, more stability and higher accuracy than

the PINN method by the results of numerical experiments.

Keywords Neural network · Ivancevic option pricing · Black–Scholes model · Numerical solution

1 Introduction

With the rapid development of the economy and society, the application of mathematics in the economic field has become more and more extensive. In the nineteenth century, PDEs developed rapidly with the deepening of research on mathematical and physical problems. Mathematical models built on the background of practical problems in physics, mechanics, medicine, finance, transportation, etc., can be transformed into differential equations to describe. Therefore, many practical problems are closely related to the solution of differential equations. The solution of differential equations can provide necessary data support for engineering design, which can ensure that the project can be completed safely, effectively, reliably and economically.

In recent years, there have been many traditional methods for solving partial differential equations, such as Lie symmetry method [1], Bäcklund transformation method [2], Darboux transformation method [3], Hirota bilinear method [4], Painleve analysis method [5], F-expansion method [6], Jacobi function expansion method [7], Homogeneous balance method [8], and

Y. Bai (✉)
College of Information Engineering, Shanghai Maritime University, Shanghai 201306, China
e-mail: baiyuxingsd@163.com

T. Chaolu
College of Arts and Sciences, Shanghai Maritime University, Shanghai 201306, China

S. Bilige
Department of Mathematics, Inner Mongolia University of Technology, Hohhot 010051, China

Tanh function method [9]. Although these methods can construct exact solutions of differential equations, the calculation process is complicated and difficult. In particular, the order of differential equations and the number of nonlinear terms will increase the difficulty of solving differential equations. In addition, there are also many exact solutions of differential equations that are difficult to solve by traditional methods, so domestic and foreign scholars have invested a lot of time and energy to study the problem of solving differential equations. In the past ten years, with the development and application of science and technology such as face recognition, medical care, voice recognition, and autonomous driving, human life has been greatly improved. The successful application of these technologies is attributed to the development of neural network [10–14]. Recently, it has been proved that the physical information neural network (PINN) can solve the forward and inverse problems of PDEs successfully, and some scholars have made good achievements by using this method, such as the PINN method has been used for solving the forward and inverse problems of Schrödinger equation, Allen Cahn equation, Navier–Stokes equation and Korteweg–de Vries equation in mathematical physics by Raissi et al. [15]. Costabal et al. [16] used the PINN method to solve the nonlinear wave dynamics equation that is satisfied by the active time when the heart is activated in 2020.

However, as the complexity of the differential equation itself or its solution increases, the PINN network has been slow to train and does not converge. These factors have an adverse effect on the performance of the network. Therefore, it is very important to design an efficient and adaptable algorithm without sacrificing network performance. As we all know, two important characteristics of improving the error accuracy of neural network are the selection of activation function and loss function, which inspires us to make two improvements in the network. Firstly, the local adaptive activation function [17, 18] is introduced, that is, a scalable parameter is introduced for each neuron in each hidden layer (this parameter is usually called the activation function slope in mathematics), these parameters can be optimized by any optimization method, which can improve the performance of the network; secondly, a slope recovery term is added to the loss function, which further accelerates the convergence speed and reduces the training cost. We named the improved method the IPINN method.

At present, many scholars have used deep learning to study the numerical solutions of many differential equations, and most of the research objects are soliton solutions and traveling wave solutions. There are very few researchers studying the rogue wave solutions of differential equations. In order to verify the effectiveness of the IPINN method, we studied two classic models in finance, namely the Ivancevic option pricing model [19–22] and the Black–Scholes model [23]. This paper uses IPINN to obtain the financial rogue waves in the Ivancevic nonlinear option pricing model. These rogue wave solutions can be used to describe the possible physical mechanism of the rogue wave phenomenon in the financial market and related fields. Meanwhile, the numerical solution of the Black–Scholes model is also obtained. As far as we know, this is the first time that we have used the IPINN method to study the Ivancevic option pricing model's financial rogue wave solution, soliton solution, and the numerical solution of European call options and put options of the Black–Scholes model. At the same time, we also use the PINN method to solve the above problems. Comparing the results of the two methods, we can see that the IPINN method has better performance.

This article is organized as follows. Section 2 introduces the IPINN algorithm; Sect. 3 is divided into 3 subsections, and Sect. 3.1 presents the financial rogue wave solution in the Ivancevic nonlinear option pricing model by the IPINN, Sect. 3.2 provides the soliton solution of Ivancevic option pricing model; Sect. 3.3 studies the numerical solutions of European call options and put options of the Black–Scholes model by this approach; Finally, summarizes our research results in Sect. 4. All the codes in this article are based on Python 3.7.3 and Tensorflow 1.14.0. All numerical experiments reported here are running on the laptop with the 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30 GHz processor and 16.0 GB of memory.

2 Methodology

2.1 The PINN method

In this section, we aim to introduce the PINN algorithm. We use a neural network with a depth of L . The network has $L - 1$ hidden layers, an input layer, and an output layer. Each hidden layer has n_l neurons. The output value of each layer will be used as the input value of the

next layer, and the corresponding affine transformation will be described below.

$$\begin{aligned}\mathcal{L}_l(\mathbf{x}^{l-1}) &= \mathbf{W}^{(l)}\mathbf{x}^{l-1} + \mathbf{b}^{(l)}, l = 1, \dots, L, \\ U(\mathbf{x}; \theta) &= (\mathcal{L}_L \circ \sigma \circ \mathcal{L}_{L-1} \circ \dots \circ \sigma \circ \mathcal{L}_1)(\mathbf{x}^0),\end{aligned}\quad (1)$$

where θ represents the trainable parameters consists of $\{\mathbf{W}^l, \mathbf{b}^l\}_{l=1}^L$ and it means the weights from layer $l-1$ to layer l , $\mathbf{w}^l \in \mathbb{R}^{n_l \times n_{l-1}}$ and bias term $\mathbf{b}^k \in \mathbb{R}^{n_l}$, $\mathbf{x}^0$ is input vector. σ is activation function and $\circ$ is the composition operator. $U(\mathbf{x}; \theta)$ is the predicted solution.

We will consider a type of PDE with the following characteristics

$$u_t + \mathcal{N}(x, u, u_x, u_{xx}, \dots) = 0, x \in \Omega, t \in [0, T], \quad (2)$$

and define the $\mathcal{F}(t, x)$

$$\mathcal{F}(t, x) := u_t + \mathcal{N}(x, u, u_x, u_{xx}, \dots). \quad (3)$$

$\mathcal{F}(t, x)$ represents the residual of the network and it is a physical constraint with regularization. $\mathcal{N}$ is a non-linear function which is composed of independent variables x, t , dependent variable u and the derivatives of u . In order to obtain the predicted solution u of Eq. (2), we need to define a loss function to ensure that the optimal parameters are obtained in the optimization process, so that the predicted solution is more accurate. Equation (3), the initial and boundary value conditions are embed into the loss function, namely

$$\begin{aligned}\text{Loss}_\theta &= \text{Loss}_u + \text{Loss}_{\mathcal{F}_u} \\ \text{Loss}_u &= \frac{1}{N_u} \sum_{i=1}^{N_u} \|U(t_u^i, x_u^i) - u(t_u^i, x_u^i)\|^2, \\ \text{Loss}_{\mathcal{F}_u} &= \frac{1}{N_f} \sum_{i=1}^{N_f} \|\mathcal{F}(t_f^i, x_f^i)\|^2.\end{aligned}\quad (4)$$

where Loss_u is the error between the predicted solution and the exact solution satisfying the initial boundary value condition. t_u^i, x_u^i represent the point in the feasible region that satisfies the initial boundary condition, $U(t_u^i, x_u^i)$ means the predicted solution of $u(t_u^i, x_u^i)$ that satisfies the initial and boundary value conditions, N_u is the number of sampling points on the initial value and boundary value data. $\text{Loss}_{\mathcal{F}_u}$ represents the residual of the network, and t_f^i, x_f^i were collocation points in the feasible region obtained by the Latin hypercube sampling (LHS) strategy [24], N_f is the number of

sampling points. In order to minimize the loss function, the Adam (i.e., a method for Stochastic Optimization) [25] and L-BFGS (i.e., a full batch gradient descent optimization algorithm based on the quasi-Newton method) [26] method are employed to optimize the parameter weight $\mathbf{W}$, biases $\mathbf{b}$. Finally, the predicted solution $U(\mathbf{x}; \theta)$ is obtained by the PINN.

2.2 The IPINN method

In this section, we aim to introduce the IPINN algorithm. We use a neural network with a depth of L . The network has $L-1$ hidden layers, an input layer, and an output layer. Each hidden layer has n_l neurons. Select the adaptive activation function, and use the activation function to act on each component of the transformed vector. The output value of each layer will be used as the input value of the next layer, and the corresponding affine transformation will be described below.

$$\begin{aligned}\mathcal{L}_l(\mathbf{x}^{l-1}) &= \mathbf{W}^{(l)}\mathbf{x}^{l-1} + \mathbf{b}^{(l)}, l = 1, \dots, L, \\ \mathcal{U}(\mathbf{x}; \Theta) &= (\mathcal{L}_L \circ \sigma \circ na_i^{L-1}(\mathcal{L}_{L-1})_i \\ &\quad \circ \dots \circ \sigma \circ na_i^1(\mathcal{L}_1)_i)(\mathbf{x}^0), \\ na_i^l &= 1, \forall n \geq 1\end{aligned}\quad (5)$$

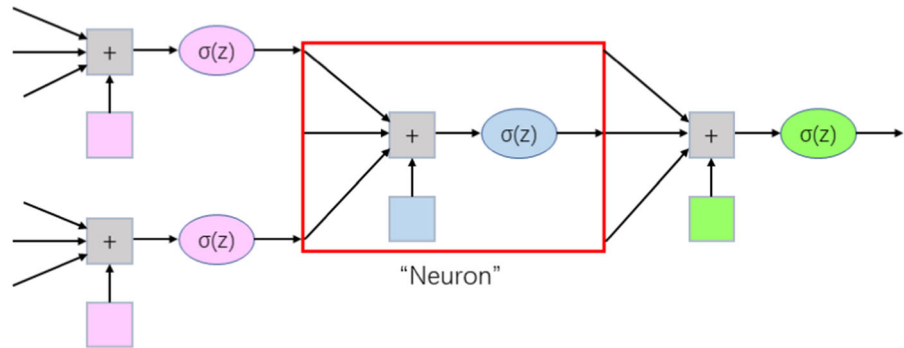
where Θ represents the trainable parameters consists of $\{\mathbf{W}^l, \mathbf{b}^l\}_{l=1}^L$ and it means the weights from layer $l-1$ to layer l , $\{\mathbf{a}_i^l\}_{l=1}^{L-1}$, $i = 1, 2, \dots, n_l$. $\mathbf{w}^l \in \mathbb{R}^{n_l \times n_{l-1}}$ and bias term $\mathbf{b}^k \in \mathbb{R}^{n_l}$, $\mathbf{x}^0$ is input vector. We also give the architecture of a neural network in Fig. 1. a_i^l is the slope of the activation function which makes each neuron have its own slope of the activation function, the slope of the activation function can be changed by optimizing the loss function, and the optimal slope of the entire network is obtained finally. n is a scaling factor which makes the optimization algorithm more sensitive. σ is activation function, and we use the hyperbolic tangent function as the activation function in this paper, and $\circ$ is the composition operator. $\mathcal{U}(\mathbf{x}; \Theta)$ represents the predicted value of the IPINN.

We will consider a type of PDE with the following characteristics

$$u_t + \mathcal{N}(x, u, u_x, u_{xx}, \dots) = 0, x \in \Omega, t \in [0, T], \quad (6)$$

and define the $\mathcal{F}(t, x)$

$$\mathcal{F}(t, x) := u_t + \mathcal{N}(x, u, u_x, u_{xx}, \dots). \quad (7)$$

Fig. 1 Neural network

$\mathcal{F}$ represents the residual of the network, and it is a physical constraint with regularization. $\mathcal{N}$ is a non-linear function which is composed of independent variables x , t , dependent variable u and the derivatives of u . In order to obtain the predicted solution $\mathcal{U}$ of Eq. (6), we need to define a loss function to ensure that the optimal parameters are obtained in the optimization process, so that the predicted solution is more accurate. We embed Eq. (7), the initial and boundary value conditions and the slope recovery term $S(a) = 1/[\frac{1}{L} \sum_{l=1}^{L-1} \exp(\frac{\sum_{i=1}^{n_l} a_l^i}{n_l})]$ into the loss function, namely

$$\begin{aligned} \text{Loss}_{\Theta} &= \text{Loss}_u + \text{Loss}_{\mathcal{F}_u} + \text{Loss}_{S(a)} \\ \text{Loss}_u &= \frac{1}{N_u} \sum_{i=1}^{N_u} \|\mathcal{U}(t_u^i, x_u^i) - u(t_u^i, x_u^i)\|^2, \\ \text{Loss}_{\mathcal{F}_u} &= \frac{1}{N_f} \sum_{i=1}^{N_f} \|\mathcal{F}(t_f^i, x_f^i)\|^2, \\ \text{Loss}_{S(a)} &= \frac{1}{\frac{1}{L} \sum_{l=1}^{L-1} \exp(\frac{\sum_{i=1}^{n_l} a_l^i}{n_l})}. \end{aligned} \quad (8)$$

where Loss_u is the error between the predicted solution and the exact solution satisfying the initial boundary value condition. t_u^i, x_u^i represent the point in the feasible region that satisfies the initial boundary condition, $\mathcal{U}(t_u^i, x_u^i)$ means the predicted solution of $u(t_u^i, x_u^i)$ that satisfies the initial and boundary value conditions, N_u is the number of sampling points on the initial value and boundary value data. $\text{Loss}_{\mathcal{F}_u}$ represents the residual of the network, and t_f^i, x_f^i were collocation points in the feasible region obtained by the Latin hypercube sampling (LHS) strategy, N_f is the number of sampling points. $\text{Loss}_{S(a)}$ makes the neural network quickly increase the activation slope value, due to the addition of this item, the gradient of the Loss function

will not disappear, and the training speed of the network is improved. In order to minimize the loss function, we employ the Adam and L-BFGS method to optimize the parameter weight $\mathbf{W}$, biases $\mathbf{b}$, and slope of the activation function a_l^i . Finally, the predicted solution $\mathcal{U}(t, x)$ is obtained.

3 Simulation results and discussions

3.1 Financial rogue wave solution for Ivancevic option pricing model

Ivancevic proposed a new nonlinear option pricing model based on Lo's modern adaptive market hypothesis theory [19] and Elliott's [21, 22] wave market theory and quantum neural computing method. This model is called the Ivancevic option pricing model.

$$i\psi_t + 0.5\sigma\psi_{SS} + \beta|\psi|^2\psi = 0, \quad (9)$$

where $\psi = \psi(S, t)$ is the option-price wave function, σ is the volatility, which can be represented by a constant or a stochastic process, $\beta = \beta(r, w)$ is the adaptive market potential. Here, we obtain financial rogue wave solution of Eq. (9) by PINN method, and the rogue wave solution may be used to describe the possible formation mechanisms in financial markets [20]. The financial rogue wave solution of Eq. (9) can be written as

$$\begin{aligned} \psi(S, t) &= \alpha \sqrt{\frac{\sigma}{2\beta}} \left[1 - \frac{4(1 + i\sigma\alpha^2 t)}{1 + 2\alpha^2(S - \sigma kt)^2 + \sigma^2\alpha^4 t^2} \right] \\ &\quad \exp\{i[kS + \frac{\sigma}{2}(\alpha^2 - k^2)t]\}, \sigma\beta > 0 \end{aligned} \quad (10)$$

where σ, β, α, k are four free parameters which control the different types of financial rogue wave propagations. To illustrate the effectiveness of the IPINN method, we let $\sigma = 1, \beta = 1, \alpha = \sqrt{2}$ and $k = 0$, and

Eq. (10) is reduced to

$$\psi(S, t) = \left[1 - \frac{4(1 + 2it)}{1 + 4S^2 + 4t^2} \right] \exp(it). \quad (11)$$

In this section, we will study the case of $\sigma = 1$, $\beta = 1$, $\alpha = \sqrt{2}$ and $k = 0$, and Ivancevic option pricing model along with Dirichlet boundary conditions can be written as

$$\begin{cases} i\psi_t + 0.5\psi_{SS} + |\psi|^2\psi = 0, S \in [S_0, S_1], t \in [t_0, t_1], \\ \psi(S, t_0) = \psi(S), \\ \psi(S_0, t) = \psi(S_1, t). \end{cases} \quad (12)$$

Substituting $t = -1.5$ into Eq. (11), the initial conditions $\psi(S) = (1 + \frac{-4+12i}{4x^2+10})\exp(-1.5i)$ can be obtained, and utilizing the linspace command in Matlab, we can get 512 and 501 equidistant nodes from the $[-2, 2]$ and $[-1.5, 1.5]$ intervals, respectively. According to Eq. (11), the experimental data are obtained by making three-dimensional graphs in Matlab. The exact solution $\psi(S) = (1 + \frac{-4+12i}{4x^2+10})\exp(-1.5i)$ is discretized into 256,512 points. Compared with the previous method of experimental data obtained by scholars (finite difference [27], Chebfun package [28]), our advantage is that the data points adopted in the experiment are accurate, not approximate data points obtained through various numerical methods, which also increases the credibility of our experiments. Specifically, $\psi(S, t)$ is the complex value and it can be formulated by $\psi(S, t) = p(S, t) + iq(S, t)$, we substitute it into Eq. (9) to get the following formulas,

$$\begin{cases} p_t + 0.5q_{SS} + (p^2 + q^2)q = 0, \\ q_t - 0.5p_{SS} - (p^2 + q^2)p = 0, \end{cases} \quad (13)$$

and define the residuals f_p, f_q as following

$$\begin{cases} f_p := p_t + 0.5q_{SS} + (p^2 + q^2)q, \\ f_q := q_t - 0.5p_{SS} - (p^2 + q^2)p. \end{cases} \quad (14)$$

Loss function is defined as $\text{Loss} = \text{Loss}_p + \text{Loss}_q + \text{Loss}_{f_p} + \text{Loss}_{f_q} + \text{Loss}_a$, where

$$\begin{aligned} \text{Loss}_p &= \frac{1}{N_\psi} \sum_{i=1}^{N_\psi} \|p(t_p^i, S_p^i) - p^i\|^2, \\ \text{Loss}_q &= \frac{1}{N_\psi} \sum_{i=1}^{N_\psi} \|q(t_q^i, S_q^i) - q^i\|^2, \end{aligned}$$

$$\begin{aligned} \text{Loss}_{f_p} &= \frac{1}{N_f} \sum_{i=1}^{N_f} \|f_p(t_{f_p}^i, S_{f_p}^i)\|^2, \\ \text{Loss}_{f_q} &= \frac{1}{N_f} \sum_{i=1}^{N_f} \|f_q(t_{f_q}^i, S_{f_q}^i)\|^2, \\ \text{Loss}_a &= \frac{1}{\frac{1}{L} \sum_{l=1}^{L-1} \exp(\frac{\sum_{i=1}^{n_l} a_l^i}{n_l})}. \end{aligned} \quad (15)$$

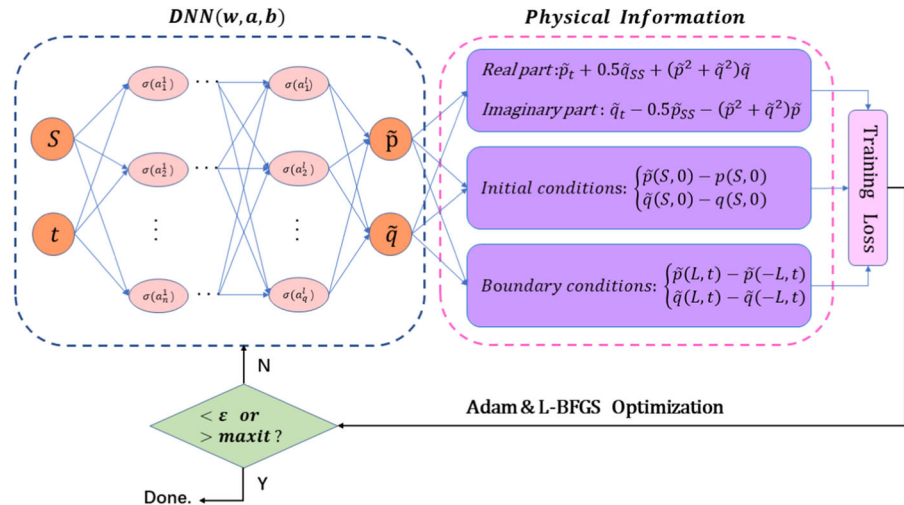
Here, $\{t_p^i, S_p^i, t_q^i, S_q^i\}_{i=1}^{N_\psi}$ denotes the collocation points on initial and boundary, $\{t_{f_p}^i, t_{f_q}^i, S_{f_p}^i, S_{f_q}^i\}_{i=1}^{N_f}$ denotes collocation points for $f_p(S, t), f_q(S, t)$.

The training points are composed of two parts. One part is that training dataset is generated in the original initial value condition and the boundary value condition by sub-sampling, randomly sample $N_\psi = 100$ points in the initial boundary value data. The other part is to select $N_f = 2000$ points by the LHS strategy. We optimize the loss function (8) by using 10,000 steps Adam and 15,000 steps L-BFGS. Schematic of IPINN for the Ivancevic option pricing model is shown in Fig. 2.

We make use of the neuron-wise locally adaptive activation functions (N-LAAF) to improve the performance of deep and physicsinformed neural networks. We initialize the scalable parameters as $n = 10, a_l^i = 0.1$ that give additional degrees of freedom to every neuron in all the hidden layers, which increases the learning capacity of the IPINN method. Xavier initialization and tanh activation function are used in IPINN method, and the financial rogue wave solution is successfully learned. In order to illustrate the advantages of IPINN, we use a neural network with a network depth of layer 9, which includes 8 hidden layers and 40 neurons in each hidden layer, to run the PINN and IPINN algorithms, respectively, under the condition of same training data $N_\psi = 100, N_f = 2000$. The relative $\mathbb{L}_2$ norm errors of $\psi(S, t)$ are $7.039245\text{e}-02$ by the IPINN method, and the relative $\mathbb{L}_2$ norm errors of $\psi(S, t)$ are $7.317705\text{e}-02$ by the PINN method.

As is shown in Fig. 3a, the financial rogue wave solution $\psi(S, t)$ was depicted by the black vertical lines at different times $t = -1.31, 0.46, 1.11$. From the point of view of the fitting effect of the predicted solution curve and the exact solution curve, we successfully captured the dynamics of the financial rogue wave. Red dashed line is predicted solution, and blue line is exact rogue wave solution of $\psi(S, t)$; (b) shows the change of the loss value with the number of itera-

Fig. 2 Schematic of IPINN for the Ivancevic option pricing model



(a)

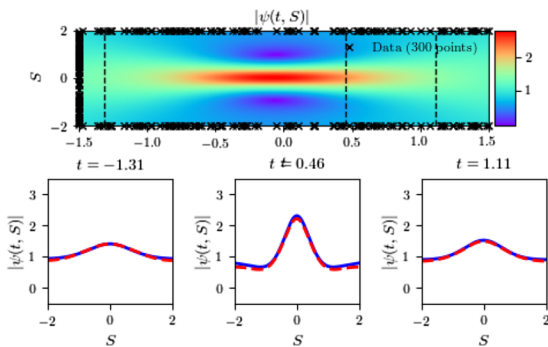
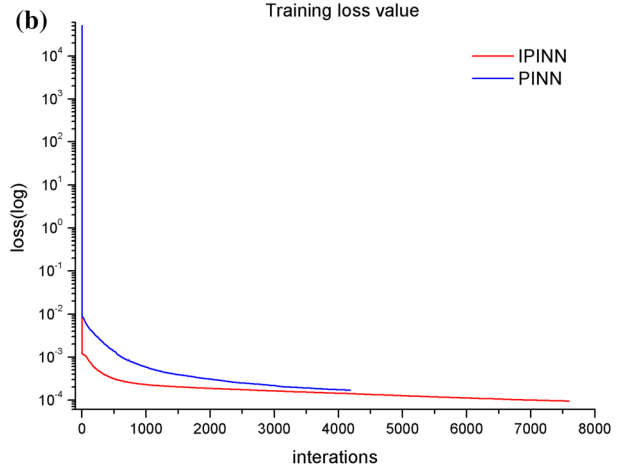


Fig. 3 Equation (12): Predicted financial rogue wave solution $\psi(S, t)$, predicted solution and exact solution at $t = -1.31, 0.46, 1.11$ were depicted by the black vertical lines in (a). Red dashed line is predicted solution and blue line is exact solution of $\psi(S, t)$; (b) shows the change of the loss value with



the number of iterations, and the vertical axis represents the logarithm of the loss value in order to better observe the change. The red line represents the IPINN method, and the blue line represents the PINN method

tions, and the vertical axis represents the base-10 logarithm of the loss value. The red line represents the IPINN method, and the blue line represents the PINN method. In order to objectively show the advantages of the IPINN method, we use the same number of network layers, neurons and the number of iterations set by the optimization loss function. We can see that the original PINN method only iterates 3931 times, and the final mean squared error loss estimations in the $\mathbb{L}_2$ norm are $1.6918175e-04$, and the IPINN method iterates 7197

times, and the final mean squared error loss estimations in the $\mathbb{L}_2$ norm are $9.445883e-05$.

From Fig. 3b, it can be seen that the IPINN method has a faster convergence speed and a higher accuracy. The results obtained from numerical experiments prove the existence of financial rogue waves. Intensity $|\psi(S, t)|$ is studied for the chosen volatility $\sigma = 1$, adaptive market potential $\beta = 1$, the scaling $\alpha = \sqrt{2}$ and the gauge $k = 0$. We can learn that time t can be chosen to be negative because the solution is invariant under the translation transformation $t \rightarrow t + t_0$ in

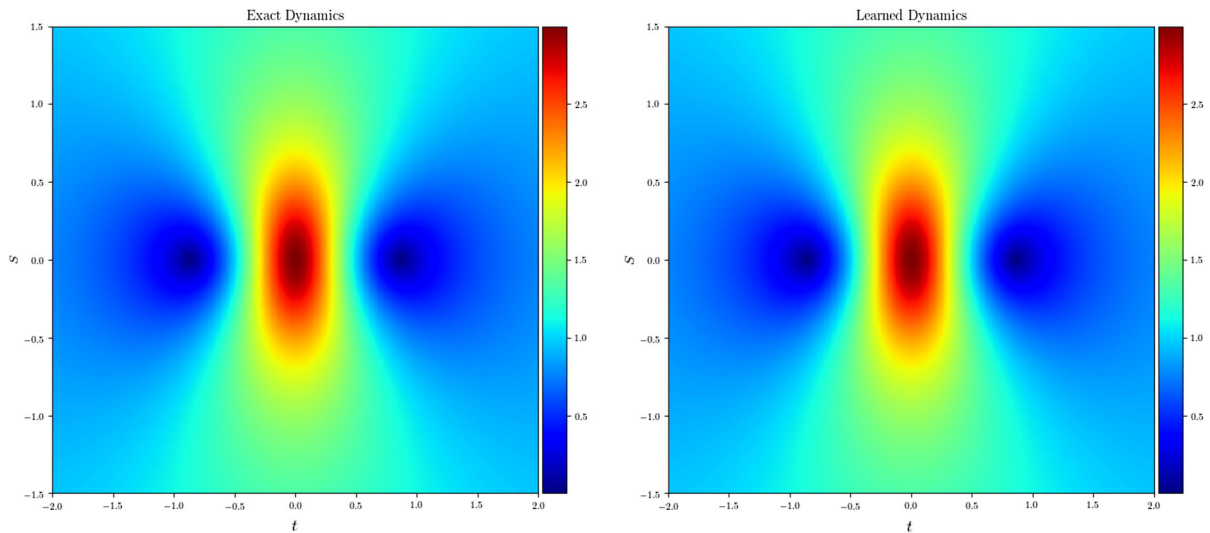


Fig. 4 The density diagrams of financial rogue wave solutions $\psi(S, t)$ of Eq. (12)

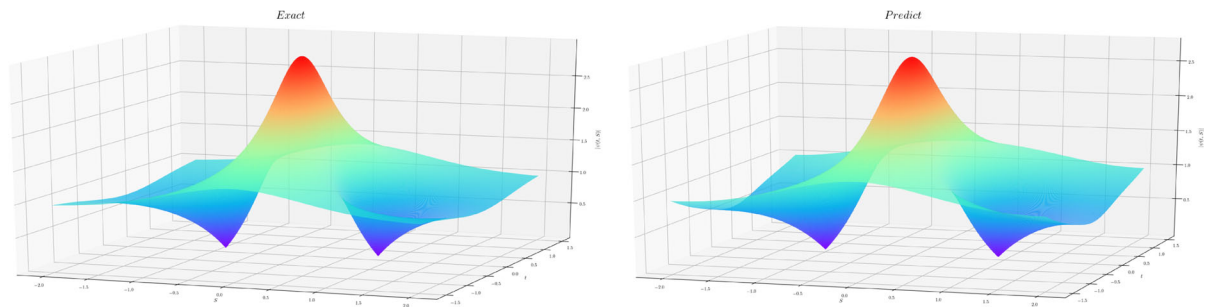


Fig. 5 Equation (12): The three-dimensional motion of the exact and predicted financial rogue wave solutions

Figs. 4 and 5. The density diagrams of financial rogue wave solutions $\psi(S, t)$ in Fig. 4, it can be seen from the density diagrams that the financial rogue wave solution has time and space symmetry. The three-dimensional motion of the exact and predicted financial rogue wave solutions is shown in Fig. 5. We had shown the exact dynamics and learned dynamics and 3D diagrams of the exact and predicted solutions of Eq. (12), it can be seen that the financial rogue wave has a hump and two troughs, intensity distributions $|\psi(S, t)|$ become more and more lower as time goes. The experimental results may further stimulate the application of financial rogue wave in financial markets and other fields related to nonlinear science. In addition, the number of layers and neurons we set at the beginning is the same as in the literature [27], we only used 2000 points for the collocation points, and he used 100,000 points, but his experimental result loss value is $7.845201\text{e}-3$,

obviously the result of our method is better than the result of the literature [27]. Relative final prediction error estimations in the $\mathbb{L}_2$ -norm for different hidden layers and different neurons per hidden layer are given in Table 1, when the number of network layers is fixed, as the number of neurons in a single layer increases, the relative error changes decreases trend. When the number of neurons in a single layer is fixed, the increase in the number of network layers has no obvious influence on the relative error. This is due to the random influence caused by some factors, and there are situations that do not fit the above conclusions. In summary, it can be made out that the network layer and the single layer of neurons jointly determine the relative final prediction error estimations in the $\mathbb{L}_2$ -norm. Table 2 exhibits the relative final prediction error estimations in the $\mathbb{L}_2$ -norm for different training points N_ψ and collocation points N_f in the case of a neural network with 9 hidden

layers and 20 neurons in each hidden layer. It can be seen from Table 2 that the influence of initial training points and configuration points on the error of $\mathbb{L}_2$ -norm is not obvious, but it can be seen that the IPINN method can select smaller initial boundary data points and relatively more configuration points to obtain numerical financial rogue wave solutions with higher accuracy.

3.2 Soliton solution for Ivancevic option pricing model

In this section, we will study soliton solution for Ivancevic option pricing model by IPINN method, and we let $\sigma = 2$, $\beta = 2$ and Ivancevic option pricing model (9) along with Dirichlet boundary conditions can be written as follow:

$$\begin{cases} i\psi_t + \psi_{SS} + 2|\psi|^2\psi = 0, & S \in [S_0, S_1], t \in [t_0, t_1], \\ \psi(S, t_0) = \psi(S), \\ \psi(S_0, t) = \psi(S_1, t). \end{cases} \quad (16)$$

and the soliton solution can be written as $\psi(S, t) = 2 \operatorname{sech}(2S + 8t) \exp(-2iS+i)$ [27]. Substituting $t = -0.5$ into Eq. (11) to get our initial conditions $\psi(S) = 2 \operatorname{sech}(2S - 4) \exp(-2iS+i)$ [27], and utilizing the linspace command in Matlab, we can get 512 and 501 equidistant nodes from the $[-5, 5]$ and $[-0.5, 0.5]$ intervals, respectively. According to Eq. (16), the experimental data are obtained by making three-dimensional graphs in Matlab. The exact solution $\psi(S, t) = 2 \operatorname{sech}(2S + 8t) \exp(-2iS+i)$ is discretized into 256,512 points. With the same data generation in Sect. 3.1, specifically, $\psi(S, t)$ is the complex value and it can be formulated by $\psi(S, t) = \hat{p}(S, t) + i\hat{q}(S, t)$, we substitute it in Eq. (9) and get the following formulas,

$$\begin{cases} \hat{p}_t + \hat{q}_{SS} + 2(\hat{p}^2 + \hat{q}^2)\hat{q} = 0, \\ \hat{q}_t - \hat{p}_{SS} - 2(\hat{p}^2 + \hat{q}^2)\hat{p} = 0, \end{cases} \quad (17)$$

and define the residuals $\hat{f}_p$ and $\hat{f}_q$ as following

$$\begin{cases} \hat{f}_p := \hat{p}_t + \hat{q}_{SS} + 2(\hat{p}^2 + \hat{q}^2)\hat{q}, \\ \hat{f}_q := \hat{q}_t - \hat{p}_{SS} - 2(\hat{p}^2 + \hat{q}^2)\hat{p}. \end{cases} \quad (18)$$

Loss function is defined as $\text{Loss} = \text{Loss}_{\hat{p}} + \text{Loss}_{\hat{q}} + \text{Loss}_{f_{\hat{p}}} + \text{Loss}_{f_{\hat{q}}} + \text{Loss}_{\hat{a}}$, where

$$\text{Loss}_{\hat{p}} = \frac{1}{N_{\psi}} \sum_{i=1}^{N_{\psi}} \|\hat{p}(t_{\hat{p}}^i, S_{\hat{p}}^i) - \hat{p}^i\|^2,$$

$$\begin{aligned} \text{Loss}_{\hat{q}} &= \frac{1}{N_{\psi}} \sum_{i=1}^{N_{\psi}} \|\hat{q}(t_{\hat{q}}^i, S_{\hat{q}}^i) - \hat{q}^i\|^2, \\ \text{Loss}_{f_{\hat{p}}} &= \frac{1}{N_{\hat{f}}} \sum_{i=1}^{N_{\hat{f}}} \|f_{\hat{p}}(t_{f_{\hat{p}}}^i, S_{f_{\hat{p}}}^i)\|^2, \\ \text{Loss}_{f_{\hat{q}}} &= \frac{1}{N_{\hat{f}}} \sum_{i=1}^{N_{\hat{f}}} \|f_{\hat{q}}(t_{f_{\hat{q}}}^i, S_{f_{\hat{q}}}^i)\|^2, \\ \text{Loss}_{\hat{a}} &= \frac{1}{\frac{1}{L} \sum_{l=1}^{L-1} \exp(\frac{\sum_{i=1}^{n_l} a_l^i}{n_l})}. \end{aligned} \quad (19)$$

Here, $\{t_{\hat{p}}^i, S_{\hat{p}}^i, t_{\hat{q}}^i, S_{\hat{q}}^i\}_{i=1}^{N_{\psi}}$ denotes the training points on initial and boundary, $\{t_{f_{\hat{p}}}^i, t_{f_{\hat{q}}}^i, S_{f_{\hat{p}}}^i, S_{f_{\hat{q}}}^i\}_{i=1}^{N_{\hat{f}}}$ denotes collocation points for $f_{\hat{p}}(S, t)$, $f_{\hat{q}}(S, t)$. The training points are obtained by randomly sample $N_{\psi} = 100$ points in the initial boundary value data and $N_{\hat{f}} = 2000$ points by the (LHS) strategy [24]. The loss function (8) is optimized by using 10,000 steps Adam and 15,000 steps L-BFGS. In order to illustrate the advantages of the improved method, we use the same number of layers as in Literature [27], and each layer has the same neurons, we use a neural network with a network depth of layer 9, 8 hidden layers and 40 neurons in each hidden layer to run the IPINN algorithms and training data $N_{\psi} = 100$, $N_{\hat{f}} = 2000$. The relative $\mathbb{L}_2$ norm errors of $\psi(S, t)$ are $2.599532\text{e}-03$, and number of iterations is 6492, and the final mean squared error loss estimations in the $\mathbb{L}_2$ norm is $5.3551015\text{e}-05$ by the IPINN method; however, the relative $\mathbb{L}_2$ norm errors of $\psi(S, t)$ are $2.566069\text{e}-2$ in the literature [27], which also shows that the IPINN method has better performance.

As is shown in Fig. 6a, the soliton solution $\psi(S, t)$ is depicted by the black vertical lines at different times $t = -0.2, 0, 0.2$. From the point of view of the fitting effect of the predicted solution curve and the accurate solution curve, we successfully captured the dynamics of the financial soliton wave and found that the financial soliton keeps its shape unchanged during the propagation process. It is concluded that the soliton in finance also has the characteristics of keeping the energy and momentum unchanged and moving over a macro-distance, so that it can transmit the absorbed energy and information without loss. Through the application of martingale method and fixed point theory, the transaction price in the financial market can be tracked effectively, which is conducive to

Table 1 Financial rogue wave solution of Eq. (12): The number of experimental training points is $N_\psi = 100$, $N_f = 2000$

$\mathbb{L}_2$ -norm error	Neurons				
	10	20	25	35	40
<i>Layers</i>					
9	7.93738e-04	2.74378e-04	1.15502e-04	8.50508e-05	9.45588e-05
10	1.63808e-04	1.52731e-04	2.02323e-04	8.39294e-05	7.44476e-05
12	6.37133e-03	7.81057e-04	1.25583e-03	3.47788e-04	1.69380e-05

Relative final prediction error estimations in the $\mathbb{L}_2$ -norm for different hidden layers and different neurons

Table 2 Financial rogue wave solutions of Eq. (12): In the case of a neural network with 9 hidden layers and 20 neurons in each hidden layer, relative final prediction error estimations in the $\mathbb{L}_2$ -norm for different training points N_ψ and collocation points N_f

$\mathbb{L}_2$ -norm error	N_ψ			
	50	100	150	200
<i>N_f</i>				
1000	6.76893e-04	8.89466e-04	5.34109e-03	7.65366e-04
2000	9.00089e-04	7.93738e-04	6.77789e-04	4.38251e-04
5000	6.85989e-03	4.62877e-04	5.41826e-04	6.45435e-04

the study of price fluctuation in the financial market. This also shows that our numerical research is meaningful. Red dashed line is predict solution, and blue line is exact soliton wave solution of $\psi(S, t)$; Fig. 6b shows the change of the loss value with the number of iterations, and the vertical axis represents the base-10 logarithm of the loss value in order to better observe the change. The density diagrams of soliton solutions $\psi(S, t)$ in Fig. 7, and the three-dimensional motion of the exact and predicted soliton solutions in Fig. 8. We had shown the exact dynamics and learned dynamics and 3D diagrams of the exact and predicted solutions of Eq. (16).

3.3 Example for Black–Scholes equation

The famous financiers Fischer Black, Myron Scholes and Rober Merton used continuous trading strategies to derive the Black–Scholes European option pricing model in 1973 under the conditions of risk-neutrality, complete market and the underlying asset price subject to geometric Brownian motion. The theoretical innovation of the market has had a huge impact, Black–Scholes model is considered to be one of the best ways to determine the fair price of options, and it also called Black–Scholes–Merton (BSM). With the improvement of Black–Scholes option pricing theory, its basic func-

tions such as risk avoidance, guaranteed return, and price monitoring provide protection for investment. In this section, we will use the IPINN method to study the Black–Scholes differential equations [23],

$$V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} - rV + rSV_S = 0,$$

$$V(S, 0) = \max(S - K, 0), \quad (20)$$

The Black–Scholes formula to obtain option price is the solution of Eq. (20) and can be written as follow

$$C = SN(d_1) - K \exp^{-rT} N(d_2),$$

$$d_1 = \frac{\ln(S/K) + (r + 0.5\sigma^2)T}{\sigma T},$$

$$d_2 = d_1 - \sigma\sqrt{T}. \quad (21)$$

where C denotes Call option price, S denotes Stock price, K denotes Strike price, r denotes Risk-free interest rate, T denotes Time to maturity, σ denotes Volatility, N denotes Standard normal distribution. If we let $r = 0.1$, $\sigma = 0.05$, $K = 10$, $S \in [0, 20]$, $t \in [0, 1]$, we use the classic finite difference method in Python to simulate Eq. (20) to obtain training data, and take 200 points equidistantly on the interval $[0, 20]$, and take 100 points equidistantly on the interval $[0, 1]$. In order to demonstrate the effectiveness of our improved method, the selected network has the same number of network layers, the same number of neurons in each hidden layer, the activation function is the hyperbolic

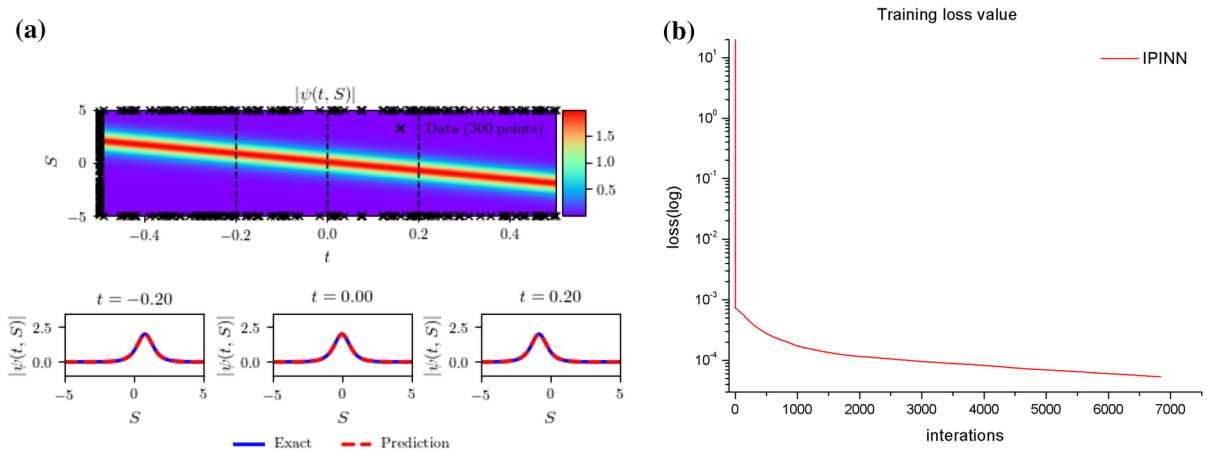


Fig. 6 Equation (16): Predicted solution $\psi(S, t)$, predicted solution and exact solution at $t = -0.2, 0, 0.2$ were depicted by the black vertical lines in (a). Red dashed line is predicted solution, and blue line is exact solution of $\psi(S, t)$; (b) shows the change

of the loss value with the number of iterations, and the vertical axis represents the logarithm of the loss value in order to better observe the change

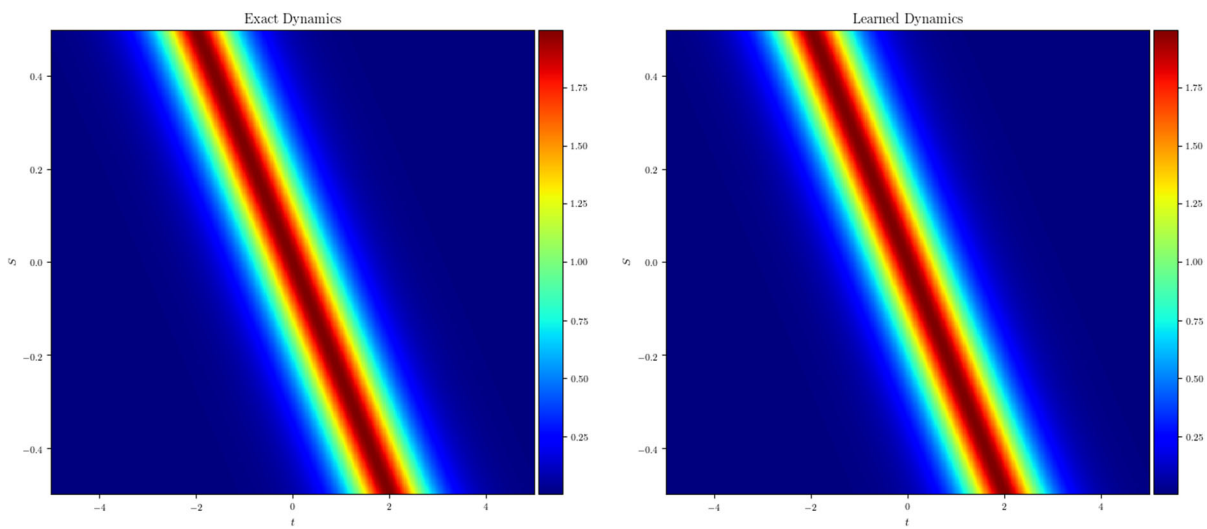


Fig. 7 The density diagrams of soliton solutions $\psi(S, t)$ of Eq. (16)

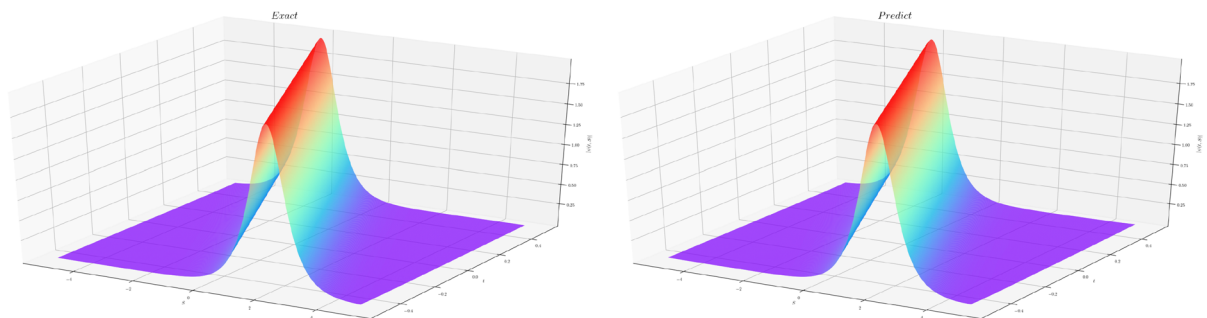


Fig. 8 Equation (16): The three-dimensional motion of the exact and predicted soliton solutions

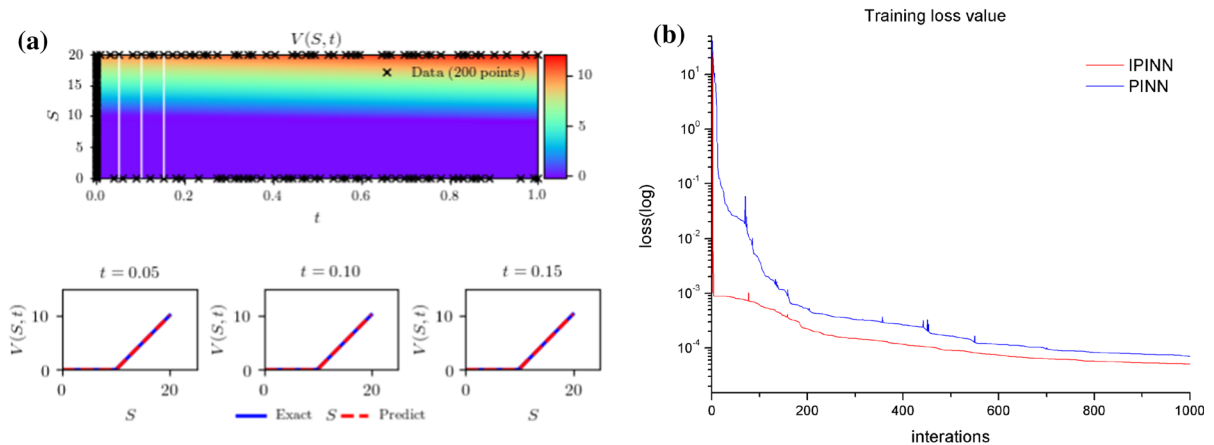


Fig. 9 Equation (20): Predicted solution $V(S, t)$ of European call options, predicted solution and exact solution at $t = 0.05, 0.1, 0.15$ were depicted by the white vertical lines in (a). Red dashed line is predicted solution, and blue line is exact solution of $V(S, t)$; (b) shows the change of the loss value with the

number of iterations, and the vertical axis represents the logarithm of the loss value in order to better observe the change. The red line represents the IPINN method, and the blue line represents the PINN method

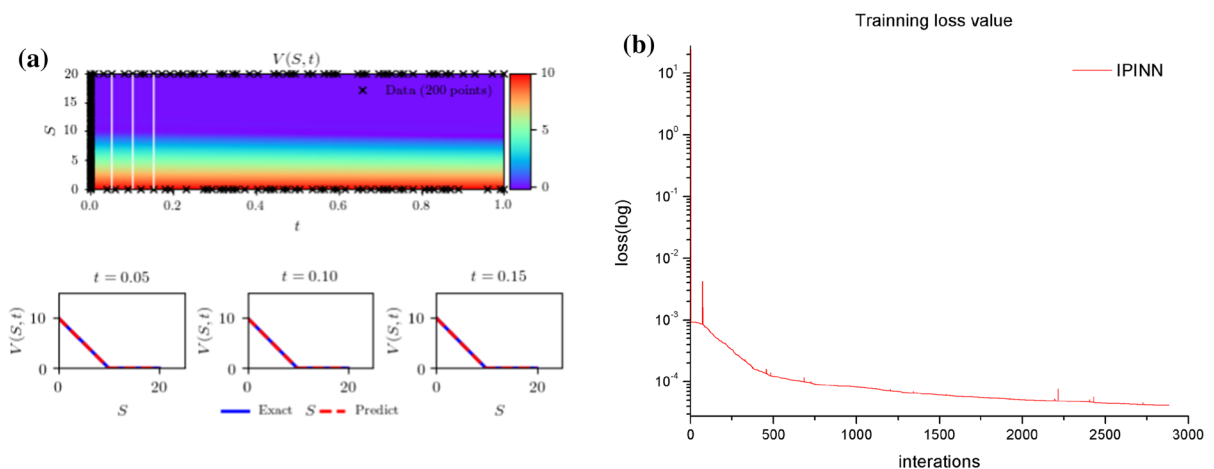


Fig. 10 Equation (20): Predicted solution $V(S, t)$ of European put options, predicted solution and exact solution at $t = 0.05, 0.1, 0.15$ were depicted by the white vertical lines in (a). Red dashed line is predicted solution, and blue line is exact solution of $V(S, t)$; (b) shows the change of the loss value with the

number of iterations, and the vertical axis represents the logarithm of the loss value in order to better observe the change

tangent function, and the same number of optimization iterations. The IPINN method and the PINN method are used to obtain the numerical solution of European call options.

We use a neural network with a network depth of layer 9, 8 hidden layers and 50 neurons in each hidden layer to run the IPINN algorithms and training data $N_V = 200$, $N_f = 10,000$. The relative $\mathbb{L}_2$ norm

errors of $V(S, t)$ is $7.194671\text{e}-03$, and number of iterations is 1246, and the final mean squared error loss estimations in the $\mathbb{L}_2$ norm is $4.14302\text{e}-05$ by the IPINN method, but the relative $\mathbb{L}_2$ norm errors of $V(S, t)$ is $8.231892\text{e}-03$ by PINN method, which also shows that the IPINN method has better performance. As is shown in Fig. 9a, the solution $V(S, t)$ was depicted by the white vertical lines at different

times $t = 0.05, 0.1, 0.15$. In view of the fitting effect of the predicted solution curve and the accurate solution curve, we successfully captured the dynamics of the European call options. Red dashed line is predict solution, and blue line is solution of $V(S, t)$. As can be seen from Fig. 9a, for call options, as the asset price S rises, the option price $V(S, t)$ also rises accordingly, and the two maintain a positive correlation. Figure 9b shows the change of the loss value with the number of iterations by IPINN method and PINN method, and the vertical axis represents the base-10 logarithm of the loss value in order to better observe the change.

At the same time, we also use the settings of the above parameters to obtain the numerical solution of European put options by IPINN method. We use a neural network with a network depth of 9, 8 hidden layers and 50 neurons in each layer to run the IPINN algorithms and training data $N_V = 200$, $N_f = 2000$. The number of iterations is 2597, and the relative $\mathbb{L}_2$ norm errors of $V(S, t)$ are $8.494155\text{e}-03$ by IPINN method and the relative $\mathbb{L}_2$ norm errors of $V(S, t)$ is $9.550976\text{e}-03$ by PINN method. We, respectively, give a comparison between the exact solution and the predicted solution of European put options (see Fig. 10a), and it can be seen that for put options, the asset price S is negatively correlated with the option price $V(S, t)$. At the same time, the experimental data show that the IPINN method is an effective neural network algorithm for solving European option pricing problems. The change of the loss value with the number of iterations (see Fig. 10b). Since the numerical solution of the put option is similar to the numerical solution of the call option, we will not repeat it here.

4 Conclusions

In recent years, the PINN method has caused a research boom in the intersection of machine learning and computational mathematics, and many scholars have used the PINN method to study the forward and inverse problems of differential equations. In this paper, the PINN method is improved by introducing a local adaptive activation function.

The innovation of this paper is the first successful application of the IPINN method to the Ivancevic option pricing model and the Black–Scholes model in the financial field, respectively, to study the soliton solution and the strange wave solution of the Ivance-

vic option pricing model and the European call options and put options of the Black–Scholes model. We use the IPINN method and the PINN method to experiment with the above models, and compare the results obtained from the two experiments. The data show that the IPINN method makes the network convergence faster and better learning performance. However, the IPINN method also has its shortcomings. This method cannot solve the differential equations with more nonlinear terms and higher orders. In particular, there are a large number of complex differential equations in mathematics, physics, engineering, and finance. With the increase in nonlinear phenomena in life, the more they are connected with practical problems, the more complex the equations become. Finding an algorithm that can solve the above problems will face more challenges, which is the direction of our later efforts.

Acknowledgements This work is supported by National Natural Science Foundation of China (Grant Nos. 11571008, 11661060, 12061054). Natural Science Foundation of Inner Mongolia Autonomous Region of China (Grant No. 2018LH01013).

Availability of data and materials The datasets generated or analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Elzehri, H., Kader, A., Latif, M.: Kink wave, dark and bright soliton solutions for complex Ginzburg–Landau equation using Lie symmetry method. *Optik* **241**, 167048 (2021)
2. Liu, P., Cheng, J., Ren, B., et al.: Bäcklund transformations, consistent Riccati expansion solvability, and soliton-cnoidal interaction wave solutions of Kadomtsev–Petviashvili equation. *Chin. Phys. B* **29**, 110–118 (2020)
3. Yang, D.Y., Tian, B., Qu, Q.X., et al.: Generalized Darboux transformation and the higher-order semirational solutions for a nonlinear Schrödinger system in a birefringent fiber. *Mod. Phys. Lett. B* **34**, 2150013 (2021)
4. Yu, J.P., Ma, W.X., Khalique, C.M., et al.: Localized interaction solution and its dynamics of the extended Hirota–Satsuma–Ito equation. *Mod. Phys. Lett. B* **35**, 2150313 (2021)
5. Farah, N., Seadawy, A.R., Ahmad, S., et al.: Interaction properties of soliton molecules and Painlevé analysis for nano bioelectronics transmission model. *Opt. Quantum Electron* **52**, 329 (2020)

6. Abdou, M.A.: The extended F-expansion method and its application for a class of nonlinear evolution equations. *Chaos Solitons Fractals* **31**, 95–104 (2007)
7. Zhang, J., Hao, H.Q.: Soliton solutions of the AB system via the Jacobi elliptic function expansion method. *Optik* **244**, 167541 (2021)
8. Radha, B., Duraisamy, C.: The homogeneous balance method and its applications for finding the exact solutions for nonlinear equations. *J. Ambient Intell. Human. Comput.* **6**, 1–7 (2020)
9. Mca, B., Io, A., As, A., et al.: Optical solitons of the (2 + 1) dimensional Biswas–Milovic equation using modified extended tanh-function method. *Optik* **245**, 167631 (2021)
10. He, Q.Z., Tartakovsky, A.M.: Physics-informed neural network method for forward and backward advection-dispersion equations. *arXiv print* [arXiv:2012.11658](https://arxiv.org/abs/2012.11658) (2021)
11. Shashank, R.V., Sai, N.B.: Physics-informed neural network method for solving one-dimensional advection equation using PyTorch. *arXiv print* [arXiv: 2103.09662](https://arxiv.org/abs/2103.09662) (2021)
12. Ali, M., Rahman, M.L., Chaki, J.: Machine translation using deep learning for universal networking language based on their structure. *Int. J. Mach. Learn. Cybern.* **3**, 1–12 (2021)
13. Bai, Y., Chaolu, T., Bilige, S.: Physics informed by deep learning: numerical solutions of modified Korteweg–de Vries equation. *Adv. Math. Phys.* **2021**, 1–11 (2021)
14. Raissi, M., Karniadakis, G.E.: Hidden physics models: machine learning of nonlinear partial differential equations. *J. Comput. Phys.* **357**, 125–141 (2018)
15. Raissia, M., Perdikaris, P., Karniadakisa, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)
16. Hurtado, D.E., Costabal, F.S., Yang, Y., et al.: Physics-informed neural networks for cardiac activation mapping. *Front. Phys.* **357**, 42 (2020)
17. Jagtap, A.D., Kawaguchi, K., Karniadakis, G.E.: Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proc. R. Soc. A Math. Phys. Eng. Sci.* (2020). <https://doi.org/10.1098/rspa.2020.0334>
18. Jagtap, A.D., Kawaguchi, K., Karniadakis, G.E.: Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proc. R. Soc. A Math. Phys. Eng. Sci.* **476**, 20200334 (2020)
19. Lo, A.: The adaptive markets hypothesis: market efficiency from an evolutionary perspective. *J. Portf. Manag.* **30**, 15–29 (2004)
20. Yan, Z.Y.: Financial rogue waves. *Commun. Theor. Phys.* **11**, 947–949 (2010)
21. Frost, A.J., Prechter, R.R., Collins, C.J., et al.: Elliott wave principle: key to stock market profits. *New Classics Library* (1978)
22. Poser, S.W., Plummer, P.J.: *Applying Elliot Wave Theory Profitably* (2003)
23. Black, F., Scholes, M.: The pricing of options and corporate liabilities. *J. Polit. Econ.* **81**, 637–654 (1973)
24. Stein, M.: Large sample properties of simulations using latin hypercube sampling. *Technometrics* **29**, 143–151 (1987)
25. Kingma, D., Ba, J.: Adam: A Method for Stochastic Optimization. *Computer Science* (2014)
26. Liu, D.C., Nocedal, J.: On the limited memory BFGS method for large scale optimization. *Math. Program.* **45**, 503–528 (1989)
27. Pu, J., Li, J., Chen, Y.: Soliton, breather and rogue wave solutions for solving the nonlinear Schrödinger equation using a deep learning method with physical constraints. *Chin. Phys. B* **6**, 77–87 (2021)
28. Driscoll, T.A., Hale, N., Trefethen, L.N.: *Chebfun Guide*. Pafnuty Publications, Oxford (2014)
29. Teng, X., Zhang, W.: Qualitative analysis and exact solutions of Huxley equation. *Appl. Math. Ser. B.* **21**, 65–69 (2006)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.